MAGI

A CVAE-based generative particle source for Geant4

User Manual — v0.8.2-beta

Francesco Monastra
INAF — francesco.monastra@inaf.it

July 30, 2026

MAGI v0.8.2-beta reproduces the *joint* phase-space distribution (energy–position–direction correlations) and the energy continuum to within its stated acceptance bars, on three seeds and two reference sources. It does **not** yet reproduce spectral-line *intensities*: measured per-line recovery runs from 0.7× to 4.9× the true value and is unstable across seeds. Line *positions* are reliable to ≤ 0.5 eV.
Read Section 8 before quoting any number derived from a MAGI-generated source. Do not use this release to estimate a fluorescence or annihilation line flux.

Contents

1 Introduction

1.1 What MAGI is

MAGI is a Conditional Variational Autoencoder (CVAE) [1, 2], implemented in Keras/TensorFlow, that learns the phase-space distribution of particles crossing a detector surface in a Geant4 [3, 4] Monte Carlo simulation — their energy, position, direction and particle type — and then samples new, physically consistent events at a small fraction of the cost of running full Geant4 transport.

The package is installed and imported as `magi`.

1.2 What it is for

The intended use is as a **drop-in replacement for a Geant4 particle source**. Run the expensive full-physics simulation once to obtain a reference sample of particles crossing some surface; train MAGI on that sample; then use MAGI's output as the source for downstream simulations that would otherwise require repeating the expensive upstream physics.

This pays off when

- large numbers of particles are needed downstream;
- the events reaching the surface of interest are rare relative to the number of primaries thrown, so direct simulation is statistics-starved;
- the same downstream simulation is run many times with different geometry or configuration choices, and re-deriving the upstream flux each time is wasteful.

1.3 What it is not for

- Estimating the intensity of an individual spectral line (Section 8).
- Detector geometries other than a sphere. The crossing surface is hard-coded as a sphere; see Section 9.
- Extrapolating outside the energy range or particle mix of the training sample. MAGI is a density model of the data it was shown, not a physics engine.

1.4 How to read this manual

Section 3 describes the code and what each module is for. Section 4 is the API reference: functions, their settings and the contracts between them. Section 5 covers the command-line tools around a run. Sections 6 and 7 are worked examples of a full run and of a validation. Section 8 states what has actually been measured, and Section 9 lists the traps.

If you are new, start with `Example_Usage.ipynb` at the repository root: it is a runnable, commented walkthrough of the whole pipeline on synthetic data, and it finishes in a few minutes.

> Energies are in MeV and positions in mm throughout, matching Geant4's own output convention. Code that must run on the CPU is shown with the `CUDA_VISIBLE_DEVICES=-1` prefix, which must be set *before* `magi` is imported.

2 Theoretical foundation

This section states what MAGI is built on and where each piece is implemented. It is background, not a derivation; the primary sources are cited so a reader can go to them directly.

The entries in `docs/manual/magi_theory.bib` are marked `% VERIFY`: their arXiv identifiers and titles are believed correct, but no author has yet opened each source to confirm author lists, venue, volume and pages. Confirm them on NASA ADS, INSPIRE or DBLP before any of this text is reused in a citable document.

2.1 Conditional variational autoencoders

A variational autoencoder [1] models a density $p(x)$ through a latent variable z, training an encoder $q(z|x)$ and decoder $p(x|z)$ against the evidence lower bound

$$\mathcal{L} \;=\; \mathbb{E}_{q(z|x)}[\log p(x \mid z)] \;-\; \mathrm{KL}(q(z \mid x) \,\|\, p(z))\,. \tag{1}$$

The conditional variant [2] conditions every term on an observed covariate c, giving $q(z|x,c)$, $p(x|z,c)$ and $p(z|c)$. In MAGI c is the particle-type one-hot, which is what lets generation be steered to a chosen particle mix rather than only reproducing the training proportions.

Implemented in `magi/core/model.py`; the reconstruction and KL terms in `magi/core/losses.py`.

2.2 Normalizing flows for the energy continuum

A normalizing flow [5, 6] represents a density as an invertible, differentiable map f applied to a simple base density, with the change-of-variables formula supplying an exact likelihood:

$$\log p(y) \;=\; \log p_u(f(y)) \;+\; \log \left|\det \frac{\partial f}{\partial y}\right|. \tag{2}$$

MAGI uses monotonic rational-quadratic spline transforms [7] for the energy continuum. The choice matters here for a specific reason: the crossing spectrum of a cosmic-ray source spans roughly nine decades and contains sharp non-Gaussian structure — a Compton edge, a muon peak, a steep low-energy tail — that a single Gaussian cannot represent at all, and that an affine flow represents only poorly. A spline flow is flexible enough to follow that structure while remaining exactly invertible, which the exact-likelihood training requires.

Implemented in `magi/core/flows.py` (`ConditionalRQSFlow`), selected with `continuum_mode="flow"`.

2.3 The gated line–continuum mixture

The central modelling decision in v0.8 is that spectral line positions are *not learned*. A categorical energy head can never place a line more precisely than one bin, and on a log grid spanning nine decades a bin is hundreds of eV — orders of magnitude wider than a real fluorescence line. Positions therefore become fixed physical inputs, measured from the data and taken from evaluated atomic data, and the head becomes a mixture:

$$p(y \mid z,c) \;=\; g_0(z,c)\, p_{\mathrm{cont}}(y \mid z,c) \;+\; \sum_{\ell=1}^{L} g_\ell(z,c)\, \mathcal{N}\!\left(y;\, y_\ell,\, \sigma_\ell^2\right), \qquad \sum_{\ell=0}^{L} g_\ell = 1, \tag{3}$$

with $y = \log_{10}(E/\mathrm{MeV})$. The widths σ_ℓ are pinned to the instrument's energy resolution rather than fitted.

The gate g faces a severe class imbalance: a line can be 10^{-5} of the sample, so an unweighted cross-entropy is minimized well by never routing anything to it. MAGI therefore applies focal down-weighting of easy examples [8], exposed as `gate_focal_gamma`.

Use $\gamma = 1$. At $\gamma \geq 2$ the gate over-corrects and begins routing genuine continuum events into the line components, visibly digging a hole in the continuum on either side of each line. This

is measured, not assumed: the near-line continuum ratio degrades monotonically across a γ sweep.

2.4 The learned conditional prior

With a standard $p(z) = \mathcal{N}(0, I)$, two problems arise. First, the decoder alone must carry every correlation between energy and geometry. Second, and more subtly, the model is trained on $z \sim q(z|x)$ but generates from $z \sim p(z)$; when the aggregated posterior does not match the prior, the decoder is evaluated at generation time in regions of latent space it was never trained on [9]. On the reference sources this gap is large — validation KL of 21–28 nats at `latent_dim` = 8.

MAGI replaces the fixed prior with a learned conditional coupling flow $p(z \mid c)$, built from affine coupling layers [10] and trained with a Monte-Carlo estimate of the KL term. This is what brings the coupling residuals of Section 8 inside their bar, and it is the difference the v0.7.2 categorical head cannot make up.

In v0.8.2 the prior's conditioning is widened from the particle-type one-hot alone to $[\,c,$ zone$\,]$, where zone is the continuum/line routing target (`prior_zone_conditioning=True`). Without the zone in the conditioning, the prior has no way to express which mixture component an event belongs to, so at generation time rare lines are routed by whatever the type marginal happens to imply.

Implemented in `magi/core/priors.py` (`ConditionalCouplingPrior`).

2.5 Atomic transition energies

Line positions are only as good as the table they are pinned at. MAGI takes transition *labels* (Kα_1, Kβ, and so on) from Bearden's compilation [11], but transition *energies* from the EADL evaluated library [12], because EADL is what the Geant4 physics list actually emitted.

This distinction is not pedantic. Pinning to Bearden energies while the simulation emits EADL displaces Cu Kα_1 by 42 eV and Cu Kβ by 40 eV — around ten times the 4 eV instrument FWHM the generated lines are given. The generated and real lines then have essentially no overlap, and no amount of gate tuning can recover it. Every pinned position is audited against a centroid measured from the real spectrum (`tools/line_centroid_audit.py`).

2.6 Optimization

Training uses Adam [13] with a task-adaptive loss-weight scheduler (`magi/training/adaptive_callbacks.py`) that rebalances the per-variable loss terms during training, so that no single head dominates the objective purely because its natural scale is larger.

2.7 Relation to other work

MAGI's problem — learn a compact, resampleable particle source on a surface so that a multi-stage Monte Carlo can be decoupled — is the same one KDSource [14] addresses with kernel density estimation. MAGI substitutes a deep conditional generative model, which scales better to the joint energy–position–direction–type density and conditions naturally on particle type.

The broader use of deep generative models to replace expensive detector simulation is well established in high-energy physics [15], with GAN- [16] and flow-based [17] calorimeter shower models now in production use [18]. MAGI differs in target: rather than showers inside a calorimeter, it models the particle flux crossing a surface, for the cryogenic X-ray instrument background problem [19].

3 Structure of the code

3.1 Repository layout

The repository has two parts: the installable library, and the experiment material that uses it.

Path	Purpose
`MAGI_package/`	The installable Python library (`import magi`). Everything in Section 4 lives here.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data. Start here.
`MAGI_v0_*.ipynb`	The experiment notebooks. Highest version number is the current pipeline; `OldNotebooks/` holds deprecated ones.
`tools/`	Command-line scripts *around* a run rather than inside one: full training runs, the acceptance harness, audits, synthetic stress tests (Section 5).
`scripts/generate_geant_source.py`	Produces a Geant4-ready source file from a trained run, outside a notebook. This is what the companion Geant4 project's `/generator/mlScript` macro invokes.
`CandidateLines/`	Candidate spectral-line tables built from a GDML mass model, in the JSON form `magi.load_candidate_energy_lines` reads.
`TrainingData/`	Cleaned, whitespace-delimited crossing data. Gitignored — large binaries stay local.
`trained_models/`, `checkpoints/`	Trained-run artifacts. Weights are gitignored; the JSON config/metadata is versioned.
`docs/`	Development notes, measurement write-ups and plans for the v0.8 line, plus this manual under `docs/manual/`.

3.2 Package layout

```
magi/
  __init__.py       public API surface (the canonical list of exports)
  magi.py           high-level convenience API: setup, build_model, train_model
  config.py         environment/seed initialization
  core/
    model.py        the CVAE classes, one per version in the lineage
    losses.py       shared NLL / angular / mixture losses
    geometry.py     physical coordinate transforms
    flows.py        conditional rational-quadratic-spline flow (v0.8 continuum)
    priors.py       conditional coupling prior p(z|cond)   (v0.8)
  data/
    io.py           load/save raw detector tables, normalization, line tables
    preprocessing.py physical features, energy binning, line detection,
                     quantile transforms, gate targets
    dataset.py      split, scale, one-hot conditioning, tf.data construction
  training/
    train.py            compile/fit wrappers
    adaptive_callbacks.py task-adaptive loss rebalancing and monitors
    checkpointing.py    save/load weights + metadata + transformers as a unit
  generation/
    sampling.py       draw latent codes and conditioning
    reconstruction.py invert model outputs back to physical quantities
    export.py         write Geant4 source files (text or chunked binary)
  validation/
    metrics.py      Wasserstein, line-integral recovery
    compare.py      histogram-residual comparisons, range/norm checks
  utils/
    plotting.py         all plot_* helpers, light/dark theme
    model_inspection.py introspect a built model
```

```
tests/              118 tests; see Section~\ref{sec:tools}
docs/USAGE.md       short-form user guide
```

3.3 The data flow

Every run follows the same seven stages. Each arrow is a function whose output is the next one's input; each stage has a matching `report_*` or `print_*` helper that prints sanity checks and returns nothing.

1. **Load** — `load_detector_table` reads the raw crossing file into a DataFrame.
2. **Physical features** — `build_physical_features` turns Cartesian position/direction into the sphere-surface parametrization $(u_r, \phi_r, u_v, \phi_v)$ and measures the primary fraction.
3. **Feature frame** — `build_feature_dataframe` bins energy, detects spectral lines and fits the quantile geometry transforms.
4. **Dataset** — `filter_particle_types_continuous_geometry` → `split_feature_data` → `scale_continuous_fea` → `build_conditioning_and_weights` → `build_tf_datasets`.
5. **Train** — a class from `magi.core`, then `compile_model` and `fit_model`.
6. **Checkpoint** — `save_final_trained_model` writes weights, config, metadata, history, task weights and a summary as one unit.
7. **Generate and validate** — `generate_latent_outputs` → `reconstruct_generated_features` → `reconstruct_generated_physics` → export, with `magi.validation` scoring the result.

3.4 Why per-variable heads

MAGI does not predict one flat output vector. Each physical variable gets its own head, because each has a different pathology:

- u_v is bounded on $[-1, 1]$ and piles up at the edges. A Gaussian head fits it badly, so the model works in $s = \operatorname{atanh}(u)$ space, or on a quantile transform of it.
- ϕ_r, ϕ_v are periodic. The v0.7 heads predict a $(\cos, \sin)$ pair softly regularized onto the unit circle; v0.7.2 predicts the angle through a quantile transform directly.
- Energy spans many decades and contains narrow spectral lines. This is the axis the version lineage is really about (Section 3.5).

3.5 Model variants

The classes in `magi.core` form a lineage. A trained run records which one it is in `model_config["model_class"]`; check that before assuming behaviour.

Class	Version	Energy / geometry
`CVAE_CatEnergy_CatUV`	v0.6	Categorical energy, discretized u_v.
`CVAE_CatEnergy_CatUV_TaskAdaptive`	v0.6	As above, plus task-adaptive loss weighting.
`CVAE_CatEnergy_ContGeom_TaskAdaptive`	v0.7	Continuous geometry, $(\cos, \sin)$ angles.
`CVAE_CatEnergy_ContPhi_TaskAdaptive`	v0.7.2	Continuous ϕ_r, ϕ_v. The stable fallback head.
`CVAE_MixEnergy_ContPhi_TaskAdaptive`	v0.8	v0.7.2 geometry; energy becomes a gated mixture of a continuum density and fixed-position line components.

The v0.8 energy head. A categorical head can never place a spectral line more precisely than one bin, and on a log grid spanning nine decades a bin is hundreds of eV wide — far broader than a real line. v0.8 therefore stops asking the model to learn line positions at all. They become *fixed physical inputs*, measured from the data, and the head becomes a mixture:

$$p(y \mid z, c) \;=\; g_0(z,c)\, p_{\text{cont}}(y \mid z, c) \;+\; \sum_{\ell=1}^{L} g_\ell(z,c)\, \mathcal{N}\!\left(y\,;\, y_\ell,\, \sigma_\ell^2\right), \tag{4}$$

where $y = \log_{10}(E/\text{MeV})$, the gate g is a softmax over $L+1$ components, the line positions y_ℓ are pinned at measured energies and the widths σ_ℓ are pinned to the instrument resolution. The continuum p_{cont} is either a single Gaussian or a conditional rational-quadratic-spline normalizing flow (`continuum_mode="flow"`).

The learned prior. With a standard $\mathcal{N}(0, I)$ prior the decoder must carry every correlation between energy and geometry on its own. Setting `prior="coupling"` replaces it with a learned conditional coupling flow $p(z \mid c)$, trained with a Monte-Carlo KL estimate. This is what makes the coupling residuals in Section 8 pass, and it is v0.8's main scientific contribution.

Use `magi.print_model_structure(model)` to print a built model's generative structure, formulas, configured line table and parameter counts.

4 Usage: functions and settings

Every exported function carries a full NumPy-style docstring, so hovering over a name in VSCode shows its parameters and return value. This section covers the contracts *between* functions, which a per-function docstring cannot.

4.1 Installation

```
pip install -e MAGI_package/      # develop the library
pip install -r requirements.txt   # or install it as a dependency
```

Core dependencies: `tensorflow` (plus `tensorflow-metal` on Apple Silicon), `tensorflow-probability`, `scikit-learn`, `joblib`, `astropy`, `optuna`.

4.2 Environment setup

```
import magi
magi.initialize_environment(seed=42, cpu_only=True, quiet=True)
magi.print_tf_info()
```

`cpu_only=True` only takes effect if it runs before TensorFlow initializes its devices. In a script that imports `magi` at module level this is already too late — set the environment variable instead, before any import:

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import magi
```

Every script in `tools/` does exactly this, and for a reason: on this workload `tensorflow-metal` is slower than the CPU, and importing `tensorflow_probability` initializes the Metal device, after which `set_visible_devices` silently stops working.

4.3 Loading data

```
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)
```

Three column schemas are auto-detected from the field count on the first data line:

Cols	Schema	Adds
9	legacy	`EventId ParticleName Energy X Y Z Vx Vy Vz`
10	PrimBool-tagged	`PrimBool` (Geant4 `ParentID==0`)
13	lineage	`ParticleId`, `ParentParticleId`, `CreatorProcessName` — required for radioactive sources

4.4 Preprocessing

```
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, -507.66), radius=100.0,
    source_type="cosmic_ray",
)
magi.print_physical_summary(prep)

feature_pack = magi.build_feature_dataframe(
    prep,
    energy_binning_mode="log_fixed_count",
    n_bins=512,
    min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10",     # required by the v0.8 mixture head
)
magi.report_feature_dataframe(feature_pack)
```

`source_type` decides flux normalization. With `"cosmic_ray"` a crossing counts as primary iff `PrimBool == 1`. With `"radioactive"` it counts as primary iff `CreatorProcessName == "RadioactiveDecay"`, because for a source embedded in bulk material the literal Geant4 primary is the decaying nucleus, which never propagates — `PrimBool` is always 0. Choosing wrongly does not error; it silently yields a wrong normalization factor.

`geometry_transform` is a stringly-typed contract. The value (`"arctanh_uv_discrete"`, `"quantile_u_r_u_v"`, `"quantile_u_r_u_v_phi_r_phi_v"`) fixes the column ordering shared by preprocessing, the model's input layout and reconstruction. It is persisted in the saved metadata and **must** match between the run that trained a model and the run that generates from it. A mismatch does not raise — it produces physically wrong output.

4.5 Spectral lines

Line handling is the part of the pipeline most specific to v0.8, and the part where a silent mistake is most expensive.

Note the ordering: lines are detected on the *raw* energies, on their own fine binning, *before* `build_feature_dataframe` is called. The feature frame's binning is a separate, coarser thing — with the mixture head it no longer constrains where a line can be placed, only how the continuum is described. The resulting gate targets are then carried into the dataset as extra feature columns.

```
payload = magi.load_candidate_energy_lines("CandidateLines/cryosphere_lines.json")
candidate_lines = payload["lines"]

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)   # refine to the pinned line width
magi.print_detected_energy_lines(res)

matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)

gate_targets = magi.build_gate_targets(
    E, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution",       # the default; see the warning below
    bandwidth_fwhm_mev=4.0e-6)
line_logsigma = magi.line_logsigma_from_resolution(
    [m["candidate_energy_mev"] for m in matched], resolution_ev=4.0, fwhm=True)
```

`bandwidth_mode` controls how `build_gate_targets` decides which *real* events count as line events when building the gate's training target. It does not affect the generated line width, which is pinned separately via `line_logsigma_from_resolution`.

Raw Geant4 crossing data has **no detector response applied**, so simulated fluorescence lines are exactly monoenergetic: in the reference cosmic-ray source all 7 542 Cu Kα_1 events share one identical `float64` energy, and every nearby continuum event is a singleton. On that reasoning, using the instrument's 4 eV FWHM as a Gaussian kernel to decide line membership is the wrong tool — it hands weight to continuum a few eV away and blurs doublets closer than a few resolution widths — and `bandwidth_mode="exact"` was added to match the simulation's own ground truth.

Measured on the cosmic-ray source, it made things worse. It did not fix the Cu Kα_1 overshoot it was built for (2.011 versus 2.052, inside the noise), it regressed the previously passing Al Kα_1 from 0.959 ± 0.025 to 0.578, it regressed Cu Kβ from 1.034 ± 0.050 to 0.753, and it pushed the coupling residual from 0.0285 ± 0.0101 to 0.054, outside its bar. The tight tolerance also drove Cu Kα_2's zone probability to zero, effectively deleting it as a modelled component.

`"resolution"` is therefore the default and is what every confirmed result in Section 8 was produced with. `"exact"` remains available and tested, but it is a research option, not a recommendation.

The lesson is worth stating plainly: a training *label* that is more faithful to the simulation is not automatically a label that trains better. The resolution kernel appears to be doing useful work as a soft target, and replacing it with a hard one removed that.

`confirm_unresolved_candidate_lines` exists because a doublet closer together than one detection bin is always a single peak, so only one member is ever detected regardless of how good the matching logic is — the gap is in peak detection, not matching. It re-measures every unmatched candidate at eV resolution and confirms the ones that are real. Both `tools/run_v0_8_real.py` and `tools/acceptance_v0_8.py` must call it identically, or the checkpoint's `n_lines` silently misaligns against the rebuilt pipeline's line count.

4.6 Building the dataset

```
dataset_pack   = magi.filter_particle_types_continuous_geometry(
```

```
                    feature_pack["feat"], prob_threshold=1e-5)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                    scaled_pack, dataset_pack["n_types"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)
```

Note `scale_cols=()` for v0.7+ quantile geometry: those columns are already standardized by their own transform, and scaling them twice is a mistake the empty tuple prevents. The split is stratified on particle type; `alpha` controls how strongly rare species are up-weighted (0 = natural proportions, 1 = fully balanced, 0.5 = default).

4.7 Training

```
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_cont=4, n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y,
    line_logsigma_init=line_logsigma,
    continuum_mode="flow",
    continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
    prior="coupling",
    prior_zone_conditioning=True,
    zone_probs=zone_probs,
    gate_focal_gamma=1.0,
    w_gate_aux=2.0,
)

magi.compile_model(model, learning_rate=2e-4, optimizer="adam")
history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)
```

Setting	Default	Effect
`latent_dim`	8	Latent width. Larger widens the posterior/prior gap that the coupling prior exists to close.
`continuum_mode`	`"gaussian"`	Use `"flow"`: a single Gaussian cannot represent a multi-decade continuum.
`continuum_flow_warp`	`"affine"`	Use `"cdf"`: pre-warps the flow's input by the empirical CDF, which is what recovers the Compton edge and the low-energy tail.
`energy_flow_condition`	—	`"z_cond"` conditions the continuum flow on both latent and conditioning.
`prior`	`"gaussian"`	Use `"coupling"` for the learned conditional prior. This is what makes coupling residuals pass.
`prior_zone_conditioning`	`False`	Condition the prior on `[type, zone]` rather than type alone. Requires `zone_probs`.
`gate_focal_gamma`	0	Focal weighting on the gate loss. Use 1; $\gamma \geq 2$ digs the continuum out from between the lines.
`w_gate_aux`	—	Weight of the auxiliary gate-routing loss.

4.8 Checkpointing

```
magi.save_final_trained_model(
    model=model,
    save_dir="trained_models/my_run",
    model_name="mix_CR",
    history=history,
    model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
)
joblib.dump(transformers, "trained_models/my_run/mix_CR_quantile_transformers.joblib")
```

A run directory holds `*.weights.h5`, `*_config.json`, `*_metadata.json`, `*_history.json`, `*_task_weights.json`, `*_summary.txt` and — for quantile-geometry runs — `*_quantile_transformers.joblib`. Generation needs the config and metadata as much as the weights, because they carry the transforms that invert the preprocessing. Copy or move them together.

Always pass `model.to_generation_config()` rather than a hand-written dict. For v0.8 heads the loader *requires* every key it emits and raises on a missing one. This guard exists because the old behaviour was `model_config.get(key, default)` everywhere, and the defaults (`continuum_mode="gaussian"`, `prior="gaussian"`, `continuum_flow_warp="affine"`) select a *code path* rather than a layer shape — so a renamed key produced a silently different architecture that loaded without complaint and generated wrong output.

Checkpoints from v0.8.2 carry `config_version: 2`. Older `config_version: 1` checkpoints load unchanged: `prior_zone_conditioning` defaults to `False`, reconstructing the exact pre-existing architecture.

4.9 Generation

```
model = magi.load_task_adaptive_model_for_generation(
    save_dir="trained_models/my_run", model_name="mix_CR",
    model_config=magi.load_json("trained_models/my_run/mix_CR_config.json"),
    n_types=n_types, type_weights=type_weights, radius=100.0)

gen_pack = magi.generate_latent_outputs(
    model, n_samples=n_real, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)

reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)

gen = magi.reconstruct_generated_physics(
    reco, center=(0.0, 0.0, -507.66), radius=100.0)
```

The line-recovery metrics compare raw counts. Generating fewer events than the real sample deflates every ratio by exactly the shortfall — a 3.44× error on the reference cosmic-ray source. Set `n_samples` to the real event count when validating.

For a Geant4-ready file, use the one-call entry point, which loads the checkpoint and streams events to disk in chunks:

```
magi.generate_detector_input_file(
    save_dir="trained_models/my_run", model_name="mix_CR",
```

```
    model_config=model_config, n_types=n_types,
    type_weights=type_weights, type_probs=type_probs,
    idx_to_type=idx_to_type, geometry_metadata=geometry_metadata,
    energy_head_mode="mixture", energy_metadata=energy_metadata,
    output_file="generated/source.dat", n_events=10_000_000,
    radius=100.0, center=(0.0, 0.0, -507.66),
    output_format="binary", seed=42)
```

Neutrinos are filtered out by default (they do not contribute to detector background here). With `generate_until_n_written=True`, the default, generation continues until `n_events` *transportable* particles have been written, so Geant4 receives exactly the requested count.

5 The tools

Scripts in `tools/` operate around a run rather than inside one. All of them force CPU execution before importing `magi`. Run them from the repository root.

5.1 Training and scoring

`run_v0_8_real.py` A full training run on the real sources with live per-epoch logging and chunked generation, so a long run is observable and OOM-safe. Saves a checkpoint, spectra and a recovery report per source.

```
python tools/run_v0_8_real.py --sources CR Small --run-tag v0_8_2 \
    --epochs 40 --seed 42 --prior-zone-conditioning
```

Flags: `-sources`, `-run-tag`, `-epochs`, `-seed`, `-n-gen`, `-gen-chunk`, `-prior-zone-conditioning`, `-gate-target-bandwidth-mode` (default `exact`), `-continuum-flow-bins`, `-warp-boost-cr`.

`acceptance_v0_8.py` The pass/fail harness. It grades a trained run against explicit thresholds, so an 80-minute run is scored objectively rather than by eye. The same numbers are the paper's validation table.

```
python tools/acceptance_v0_8.py --sources CR Small --seeds 42 7 13 \
    --json docs/_data/acceptance.json
```

It reports four things: per-line integral recovery (resolution-scaled $\pm 5\sigma$ window, local continuum subtracted, normalized for $N_{\mathrm{real}}/N_{\mathrm{gen}}$); near-line continuum ratio, i.e. whether the gate dug a hole beside each line; the energy-marginal Wasserstein distance, global and per decade band; and the coupling residuals.

> The per-band Wasserstein distance on the CR source has a seed-to-seed σ/μ of 40–50 %, larger than most effects worth chasing. Two changes were adopted on single-seed evidence during development and both evaporated under a three-seed grid. Use `-seeds 42 7 13` and report mean ± std. The Small source is tighter, around 14 %.

`score_v0_7_2.py` Scores a v0.7.2 categorical-head run on the same axes, so the two heads can be compared like with like. It reads each checkpoint's own `preprocessing_metadata` rather than re-deriving it.

5.2 Audits

`line_centroid_audit.py` For each matched line, histograms the real spectrum locally at eV resolution, fits the centroid and reports |catalogue − measured| in FWHM units, failing loudly

above one FWHM. Run this before committing to a long training run: a mispositioned line cannot be fixed by any amount of training.

`build_candidate_lines_from_geant4.py` Builds the candidate-line table from a GDML mass model and Geant4's fluorescence data. It takes transition *labels* from the Bearden table (only that directory carries them) and transition *energies* from the EADL table, because EADL is what the simulation actually emitted. Getting this backwards puts every instrumental line tens of eV off — up to 10 FWHM.

`gate_posterior_prior_audit.py` Compares the gate's component fractions under $z \sim q(z|x)$ against $z \sim p(z|\text{cond})$. A large divergence localizes a line failure to the prior rather than the gate.

5.3 Synthetic stress tests

`synthetic_stress_test_coupled.py`, `_cr_multimodal.py` and `_small_singlepeak.py` build synthetic data with a known answer — an injected energy–geometry coupling, a rare-line cluster, a sparse tail. Use them to gate a change *before* spending a real run on it; they finish in minutes.

5.4 Unit tests

```
python -m pytest MAGI_package/tests/ -q
```

118 tests covering flow round-trip identity, checkpoint save/load config matching, gate-target construction (including the exact/resolution A/B), prior zone-conditioning, and public API docstring coverage. They catch mechanical regressions; they do not tell you a trained model reproduced your source.

6 Example: a full run

This is a complete run on the reference cosmic-ray source, from raw data to a Geant4 source file. On a CPU-only Apple Silicon machine it takes roughly 90 minutes for 40 epochs plus generation.

6.1 The short path

Everything below is what `tools/run_v0_8_real.py` does. In practice:

```
export CUDA_VISIBLE_DEVICES=-1
python tools/run_v0_8_real.py --sources CR --run-tag my_run \
    --epochs 40 --seed 42 --prior-zone-conditioning \
    2>&1 | tee logs/my_run_CR.log
```

6.2 The long path, step by step

```
import os
os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
import numpy as np, joblib, magi

magi.initialize_environment(seed=42, cpu_only=True)

# --- 1. load ------------------------------------------------------
df = magi.load_detector_table("TrainingData/alloutputDSCryoSphereCR.dat")
magi.report_basic_table_checks(df)

# --- 2. physical features -----------------------------------------
```

```
prep = magi.build_physical_features(
    df, center=(0.0, 0.0, -507.66), radius=100.0, source_type="cosmic_ray")
magi.print_physical_summary(prep)

# --- 3. lines: detect on the RAW energies, before the feature frame --
candidate_lines = magi.load_candidate_energy_lines(
    "CandidateLines/cryosphere_lines.json")["lines"]
E = prep["features"]["Energy"].to_numpy()

res = magi.detect_energy_lines(
    E, binning_mode="log_fixed_count", n_bins=1024,
    prominence_factor=3.0, window=5, candidate_lines=candidate_lines,
    refine_bin_width_mev=4.0e-6)
matched = [m for m in res["matched_lines"] if m["count"] >= 100]
matched += magi.confirm_unresolved_candidate_lines(
    E, candidate_lines, matched, resolution_ev=4.0)
magi.print_detected_energy_lines(res)

# --- 4. feature frame ------------------------------------------------
feature_pack = magi.build_feature_dataframe(
    prep, energy_binning_mode="log_fixed_count", n_bins=512, min_counts=20,
    geometry_transform="quantile_u_r_u_v_phi_r_phi_v",
    energy_transform="log10")          # the v0.8 head needs y = log10(E)
magi.report_feature_dataframe(feature_pack)

E_full = feature_pack["filtered_prep"]["features"]["Energy"].to_numpy()

gate_targets = magi.build_gate_targets(
    E_full, feature_pack["energy_bins"], matched,
    bandwidth_mode="resolution", bandwidth_fwhm_mev=4.0e-6)

line_positions_y = np.log10(
    [m["candidate_energy_mev"] for m in matched]).astype(np.float32)
line_logsigma = magi.line_logsigma_from_resolution(
    10.0 ** line_positions_y, resolution_ev=4.0, fwhm=True)

# --- 5. dataset: the gate targets ride along as extra feature columns -
feat = feature_pack["feat"].copy()
for j in range(gate_targets.shape[1]):
    feat[f"gate_target_{j}"] = gate_targets[:, j]
cont_cols = ("u_r_q", "u_v_q", "phi_r_q", "phi_v_q", "energy_y") + tuple(
    f"gate_target_{j}" for j in range(gate_targets.shape[1]))

dataset_pack   = magi.filter_particle_types_continuous_geometry(
                     feat=feat, prob_threshold=1e-5, cont_cols=cont_cols)
split_pack     = magi.split_feature_data(dataset_pack, random_state=42)
magi.report_split_summary(split_pack, dataset_pack["n_types"])
scaled_pack    = magi.scale_continuous_features(split_pack, scale_cols=())
condition_pack = magi.build_conditioning_and_weights(
                     scaled_pack, dataset_pack["n_types"],
                     idx_to_type=dataset_pack["idx_to_type"], alpha=0.5)
ds_pack        = magi.build_tf_datasets(condition_pack, batch_size=4096)
magi.report_tf_datasets(ds_pack)

# --- 6. per-type zone probabilities for the conditioned prior --------
n_zones   = gate_targets.shape[1]
zone_cols = dataset_pack["X_cont_raw"][:, -n_zones:]
zone_probs = np.zeros((dataset_pack["n_types"], n_zones))
for t in range(dataset_pack["n_types"]):
    mask = dataset_pack["y_type"] == t
    row  = zone_cols[mask].mean(axis=0) if mask.any() else np.zeros(n_zones)
    s    = row.sum()
    zone_probs[t] = row / s if s > 0 else np.eye(n_zones)[0]
```

```
# --- 7. model and training ---------------------------------------
model = magi.CVAE_MixEnergy_ContPhi_TaskAdaptive(
    n_cont=4, n_types=dataset_pack["n_types"], latent_dim=8,
    line_positions_y=line_positions_y, line_logsigma_init=line_logsigma,
    continuum_mode="flow", continuum_flow_warp="cdf",
    energy_flow_condition="z_cond",
    prior="coupling", prior_zone_conditioning=True, zone_probs=zone_probs,
    gate_focal_gamma=1.0, w_gate_aux=2.0)

magi.compile_model(model, learning_rate=2e-4)
magi.print_model_structure(model)

history = magi.fit_model(
    model, ds_pack["train_ds"], ds_pack["val_ds"],
    epochs=40, callbacks=magi.build_default_callbacks(), verbose=2)

# --- 8. checkpoint -----------------------------------------------
magi.save_final_trained_model(
    model=model, save_dir="trained_models/my_run", model_name="mix_CR",
    history=history, model_config=model.to_generation_config(),
    preprocessing_metadata=preprocessing_metadata,
    normalization_metadata=prep["normalization"])
```

6.3 Producing the Geant4 source file

```
python scripts/generate_geant_source.py \
    --save-dir trained_models/my_run \
    --model-name mix_CR \
    --metadata-file trained_models/my_run/mix_CR_metadata.json \
    --output-file generated/cr_source.dat \
    --n-events 10000000
```

The companion Geant4 project consumes this directly through its `/generator/mlScript` macro command.

7 Example: a validation

Training finishing is not evidence that it worked. This is how a run gets graded.

7.1 The short path

```
python tools/acceptance_v0_8.py --sources CR --run-tag my_run \
    --seeds 42 7 13 --json docs/_data/my_run_acceptance.json
```

7.2 What it checks, and against what

Criterion	Threshold	Notes
Per-line recovery	[0.8, 1.2], two-sided	Only for lines above the detection floor. Two-sided because over-generation is the dominant failure.
Per-line stability	$\sigma \leq 0.15$ over ≥ 3 seeds	So a user's single unattended run likely lands in band.
Detection floor	$\geq 5\sigma$ Poisson	The particle-physics discovery convention. Below-floor lines are *not modelled*, and their absence is a correct result.
Near-line continuum	≥ 0.85	Did the gate dig a hole beside the line?
Energy marginal	Wasserstein ≤ 0.05	Global, on $\log_{10} E$.
Coupling	$\max \lvert\Delta\text{corr}\rvert \leq 0.05$	Over $(\log E, u_r, u_v, \phi_r, \phi_v)$.

7.3 Doing it by hand

```
real = magi.reconstruct_real_test_physics(
    split_pack["X_cont_test"], split_pack["E_test_raw"],
    center=(0.0, 0.0, -507.66), radius=100.0,
    geometry_metadata=geometry_metadata)

# Generate as many events as there are real ones - see the warning in 3.9.
gen_pack = magi.generate_latent_outputs(
    model, n_samples=real["E"].size, type_probs=type_probs,
    n_types=n_types, idx_to_type=idx_to_type)
reco = magi.reconstruct_generated_features(
    gen_pack, energy_head_mode="mixture",
    geometry_metadata=geometry_metadata, energy_metadata=energy_metadata)
gen  = magi.reconstruct_generated_physics(reco, center=(0,0,-507.66), radius=100.0)

# 1. marginals
scores = magi.compute_wasserstein_scores(real, gen)
print(scores["logE"])

# 2. geometric constraints that must hold by construction
magi.report_generated_constraints(gen, radius=100.0)
magi.report_final_ranges(real, gen)

# 3. per-line recovery
recovery = magi.compute_line_integral_recovery(
    real["E"], gen["E"], matched, energy_bins,
    energy_component_idx_gen=reco["energy_component_idx_gen"],
    resolution_ev=4.0, neighbour_lines=candidate_lines)
for r in recovery:
    print(f"{r['label']:20s} {r['recovery_ratio']:6.3f}  "
          f"sig={r['line_significance']:7.1f}")

# 4. the joint distribution - this is the one that matters
cols = ["logE", "u_r", "u_v", "phi_r", "phi_v"]
df_real, df_gen, df_both = magi.build_real_generated_featureframes(real, gen)
_, corr_real = magi.plot_correlation_matrix(df_real, cols, show=False)
_, corr_gen  = magi.plot_correlation_matrix(df_gen,  cols, show=False)
print("coupling residual:", np.abs(corr_gen - corr_real).to_numpy().max())

magi.compare_hist_with_residuals(real["E"], gen["E"], "Energy [MeV]",
                                 bins=200, ratio_clip=1.0)
```

7.4 Reading the output

- **Coupling residual** is the headline. Below 0.05 means the joint distribution is reproduced, which is what MAGI is for.
- **Wasserstein on** $\log_{10} E$ below 0.05 means the spectrum's shape is right.
- `line_significance` below 5 means the line is not significantly detected in the real data either; its recovery ratio is noise and should be ignored, not reported as a failure.
- `sideband_contaminated` true means a neighbouring line sits in the continuum side-bands, so the continuum subtraction for that line is unreliable.
- A statistical floor of $\sqrt{1/n_{\text{real}} + 1/n_{\text{gen}}}$ bounds how precisely any recovery ratio can be known. For a line with a few hundred events this is already several per cent.

8 Accuracy: what has actually been measured

All numbers are mean ± std over three seeds (42/7/13) on the two reference sources, produced by `tools/acceptance_v0_8.py`.

8.1 Validated

Quantity	CryoSphere-CR	CryoSphere-Small	Bar
Coupling, max $\lvert\Delta\text{corr}\rvert$	0.0285 ± 0.0101	0.0357 ± 0.0102	≤ 0.05
Wasserstein on $\log_{10} E$	0.0241 ± 0.0046	0.0065 ± 0.0006	≤ 0.05

The joint distribution holds up with error bars: every real cross-correlation over $(\log E, u_r, u_v, \phi_r, \phi_v)$ is reproduced inside the bar, on both sources, across seeds. The v0.7.2 categorical head does not achieve this — its coupling residual is 0.226, eight times worse.

8.2 Not validated

Per-line *intensities* are wrong by factors of roughly 0.7× to 4.9×, and unstable across seeds.

Line	Recovery	Status
CR Al Kα_1	0.959 ± 0.025	in band
CR Cu Kβ	1.034 ± 0.050	in band
CR e^+e^- 511 keV	1.825 ± 0.324	over-generated, unstable
CR Cu Kα_1	2.052 ± 0.034	over-generated
Small Al Kα_2	3.412 ± 0.376	far over
Small Cu Kα_1	4.903 ± 0.356	far over

Practical consequences:

- Line *positions* are reliable — pinned physical inputs, audited to ≤ 0.5 eV against the measured spectrum on all sources. It is the number of events in each line that is wrong.
- Do not use generated output to estimate a fluorescence or annihilation line flux, or any quantity dominated by one.
- The continuum between lines, and the total energy marginal, are within the bar.
- Rare lines fare worse than strong ones, and the failure is source-dependent. Measure on *your* source with the acceptance harness rather than assuming these numbers transfer.

8.3 Choosing between the heads

Neither head is correct for lines today. v0.7.2 has a better energy marginal on the cosmic-ray source (0.0141 versus 0.0241) but produces **zero** events in the 511 keV window against 52 225

real, and its coupling is eight times worse. If your use depends on correlations, use v0.8.2. If it depends only on the marginal spectrum and you want the most conservative option, v0.7.2 remains available.

8.4 Negative results already established

Recorded so they are not re-attempted without new information:

- **Per-line gate class weights**, calibrated from measured routing: no effect distinguishable from noise. The untouched control line moved more (+27 %) than either weighted line.
- **Continuum polish** (CDF-warp knot boost, more flow bins): a three-seed grid put every difference under 1.2σ of the noise.
- **Exact-match gate labels** (`bandwidth_mode="exact"`): physically the more faithful label, but it did not fix the line it was built for and broke a confirmed result and the coupling bar. See the warning in Section 4 and `docs/v0.8.1_line_truth.md` §14.2.

9 Limitations and traps

> `build_physical_features`'s `center`/`radius` and the transforms in `core/geometry.py` all assume the crossing surface is a sphere. Adapting MAGI to a planar, cylindrical or box surface is **not** a matter of different parameters: it requires reworking the geometry-transform layer in both `preprocessing.py` and `generation/reconstruction.py`, which must stay in sync.

The training data is a crossing spectrum, not deposited energy. Events are recorded where they cross a virtual sphere, with no detector response applied. Classical detector escape peaks may legitimately be absent; a null result there is a real result.

`geometry_transform` mismatches do not raise. They produce physically wrong output. The string is persisted in metadata for this reason — pass the metadata rather than re-specifying by hand.

Particle-type filtering silently drops rare species. If a rare-but-relevant type is missing from generated output, check `prob_threshold` first.

Energy binning limits which lines can be *found*. With the v0.8 mixture head, binning no longer limits where a line can be *placed*, but on a log grid spanning many decades a bin can be hundreds of eV wide, so close doublets fall in one bin. Use `refine_bin_width_mev` and `confirm_unresolved_candidate_lines`, then verify with `tools/line_centroid_audit.py`.

Quantile transforms need enough samples. Small datasets give unreliable tail behaviour in the fitted transform, which shows up as implausible extreme values in generated output.

Flux normalization depends on `source_type`. Choosing wrongly does not error. Radioactive sources additionally need the 13-column lineage schema.

Unit tests cover machinery, not physics. 118 passing tests do not tell you a trained model reproduced your source. That is what Section 7 is for.

CPU-only by default. On this workload `tensorflow-metal` is slower than the CPU.

10 Where to look next

Document	Contents
`MAGI_package/docs/USAGE.md`	Short-form version of this manual.
`Example_Usage.ipynb`	Runnable walkthrough on synthetic data.
`docs/v0.8.1_line_truth.md`	The measurement record behind Section 8: what was tried, what it showed.
`docs/v0.8.2_RoadmapForAdoption.md`	The remaining plan and the acceptance bar as restated.
`docs/v0.8_learnable_prior_theory.md`	Why the conditional prior was needed.
`docs/v0.8_v072_comparison.md`	Head-to-head history. Note its absolute numbers predate the metric fix.

Include the run directory's `*_config.json` and `*_metadata.json`, the acceptance-harness JSON if you have it, and the seed. Most surprising results trace back to a preprocessing contract mismatch that those files make visible immediately.

References

[1] Diederik P. Kingma and Max Welling. Auto-Encoding Variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[2] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning Structured Output Representation using Deep Conditional Generative Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[3] S. Agostinelli et al. Geant4—a simulation toolkit. *Nuclear Instruments and Methods in Physics Research A*, 506(3):250–303, 2003.

[4] J. Allison et al. Recent developments in Geant4. *Nuclear Instruments and Methods in Physics Research A*, 835:186–225, 2016.

[5] George Papamakarios, Eric Nalisnick, Danilo J. Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing Flows for Probabilistic Modeling and Inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

[6] Ivan Kobyzev, Simon J.D. Prince, and Marcus A. Brubaker. Normalizing Flows: An Introduction and Review of Current Methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(11):3964–3979, 2021.

[7] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural Spline Flows. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019. Monotonic rational-quadratic spline transforms; the continuum density in magi/core/flows.py. VERIFY metadata.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal Loss for Dense Object Detection. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. Focal down-weighting of easy examples; the gate loss must route lines that are 1e-5 of the sample. VERIFY metadata.

[9] Jakub M. Tomczak and Max Welling. VAE with a VampPrior. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2018. The aggregated-posterior/prior mismatch argument for replacing N(0,I) with a learned prior. VERIFY metadata.

[10] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density Estimation using Real NVP. In *International Conference on Learning Representations (ICLR)*, 2017. Affine coupling layers; the architectural basis of the conditional prior in magi/core/priors.py. VERIFY metadata.

[11] J. A. Bearden. X-Ray Wavelengths. *Reviews of Modern Physics*, 39(1):78–124, 1967. Source of the transition labels in the candidate-line table. VERIFY metadata.

[12] S. T. Perkins, D. E. Cullen, M. H. Chen, J. H. Hubbell, J. Rathkopf, and J. Scofield. Tables and Graphs of Atomic Subshell and Relaxation Data Derived from the LLNL Evaluated Atomic Data Library (EADL), Z = 1–100. Technical Report UCRL-50400, Vol. 30, Lawrence Livermore National Laboratory, 1991. The fluorescence energies Geant4 actually emits, and therefore the ones MAGI pins its line components at. VERIFY metadata.

[13] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR)*, 2015. Default optimizer in magi.compile_model. VERIFY metadata.

[14] N. S. Schmidt, O. I. Abbate, Z. M. Prieto, J. I. Robledo, J. I. Márquez Damián, A. A. Márquez, and J. Dawidowski. KDSource, a tool for the generation of Monte Carlo particle sources using kernel density estimation. *Annals of Nuclear Energy*, 177:109309, 2022.

[15] Baran Hashemi and Claudius Krause. Deep generative models for detector signature simulation: A taxonomic review. *Reviews in Physics*, 12:100092, 2024.

[16] Michela Paganini, Luke de Oliveira, and Benjamin Nachman. CaloGAN: Simulating 3D high energy particle showers in multilayer electromagnetic calorimeters with generative adversarial networks. *Physical Review D*, 97(1):014021, 2018.

[17] Claudius Krause and David Shih. CaloFlow: Fast and Accurate Generation of Calorimeter Showers with Normalizing Flows. *Physical Review D*, 107(11):113003, 2023.

[18] ATLAS Collaboration. Deep generative models for fast photon shower simulation in ATLAS. *Computing and Software for Big Science*, 8:7, 2024.

[19] S. Lotti et al. Review of the Particle Background of the Athena X-IFU Instrument. *The Astrophysical Journal*, 909(2):111, 2021.